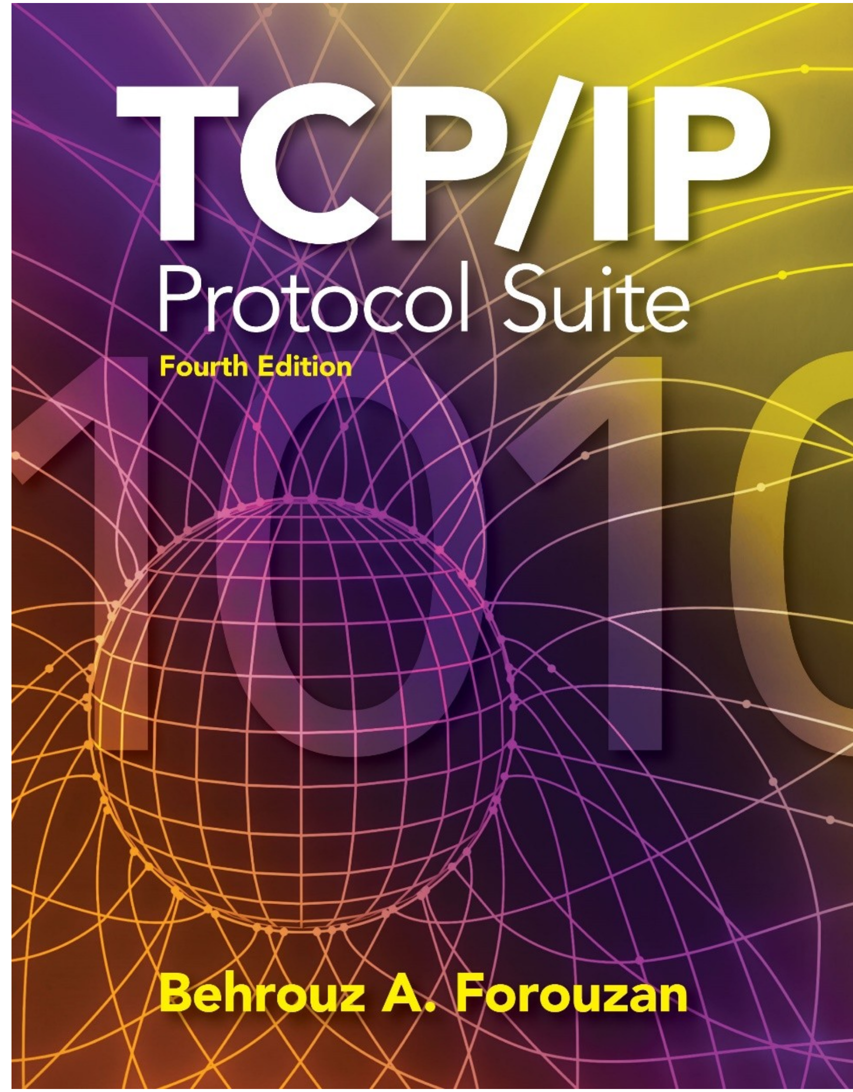
Chapter 9

Internet Control Message Protocol Version 4 (ICMPv4)



OBJECTIVES:

!

"

Chapter Outline

9-1 INTRODUCTION

The IP protocol has no error-reporting or error correcting mechanism. What happens if something goes wrong? What happens if a router must discard a datagram because it cannot find a router to the final destination, or because the time-to-live field has a zero value? These are examples of situations where an error has occurred and the IP protocol has no built-in mechanism to notify the original host.

#

$

🌐 Why ICMP Is Needed

❌ Limitations of IP

The Internet Protocol (IP) is designed to be:

- Best-effort
- Connectionless
- Unreliable

This means IP does NOT:

- Detect errors
- Correct errors
- Inform the sender when something goes wrong

So naturally, questions arise:

⚠️ What can go wrong?

Examples:

1. A router cannot find a route to the destination
2. A router drops a packet because TTL = 0
3. The destination cannot reassemble fragments (some fragments missing)
4. A host or router might be down, but the sender doesn't know
5. Network administrators need diagnostic information

➡️ IP has no built-in way to report any of these problems

✅ Solution: Internet Control Message Protocol (ICMP)

To fix these shortcomings, ICMP was created.

📌 What ICMP Does

ICMP provides:

1. Error reporting
2. Network diagnostics and queries

📌 ICMP is a companion protocol to IP, not a replacement.

🧱 ICMP's Position in the Network Layer

markdown

```
Application Layer
-----------------
Transport Layer
-----------------
Network Layer
   ├── IP
   ├── ICMP
   ├── ARP
   └── IGMP
-----------------
Data Link Layer
```

📌 ICMP is a network-layer protocol, just like IP.

📦 How ICMP Works with IP

Although ICMP is a network-layer protocol:

👉 ICMP messages are carried inside IP datagrams

📬 Encapsulation Process

mathematica

```
Frame Header
   ↓
IP Header  (Protocol field = 1)
   ↓
ICMP Message
   ↓
Frame Trailer
```

🔢 Important Detail

- IP Protocol Field = 1 → Indicates ICMP
- This tells the receiving host:

> "The payload is an ICMP message"

📌 ICMP does not bypass IP — it uses IP to travel

🚨 When ICMP Is Used (Examples)

Problem	ICMP Response
No route to destination	Destination Unreachable
TTL reaches 0	Time Exceeded
Fragment reassembly timeout	Time Exceeded
Host unreachable	Destination Unreachable
Ping request	Echo Reply

🛠 ICMP for Queries & Management

ICMP is also used for:

- Checking if a host/router is alive → `ping`
- Measuring path delay & hops → `traceroute`
- Network troubleshooting

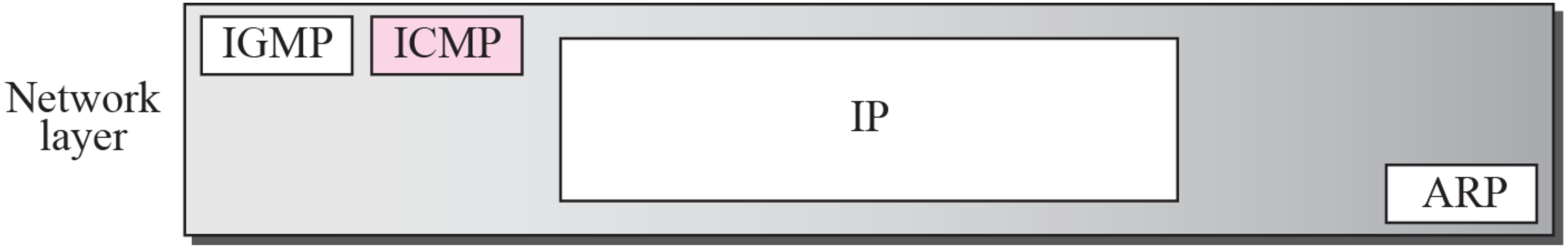
📌 Network administrators rely heavily on ICMP

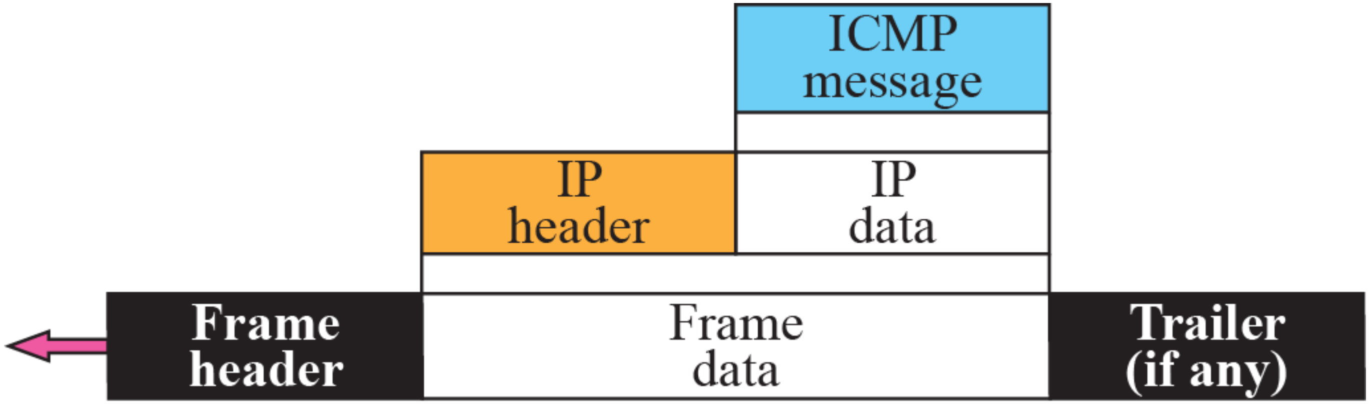
🧠 Key Exam Points to Remember

✔ IP is unreliable and silent
✔ ICMP reports errors and status information
✔ ICMP is a network-layer protocol
✔ ICMP messages are encapsulated inside IP
✔ IP Protocol field = 1 for ICMP
✔ ICMP does not correct errors, only reports

🧾 One-Line Summary (Perfect for Exams)

> ICMP compensates for IP's lack of error reporting and diagnostic capabilities by providing control and error messages, which are encapsulated inside IP datagrams.


Network layer
IGMP
ICMP
IP
ARP


ICMP message
IP header
IP data
Frame header
Frame data
Trailer (if any)

!"

ICMP messages are divided into two broad categories: error-reporting messages and query messages. The error-reporting messages report problems that a router or a host (destination) may encounter when it processes an IP packet. The query messages, which occur in pairs, help a host or a network manager get specific information from a router or another host. Also, hosts can discover and learn about routers on their network and routers can help a node redirect its messages.

%

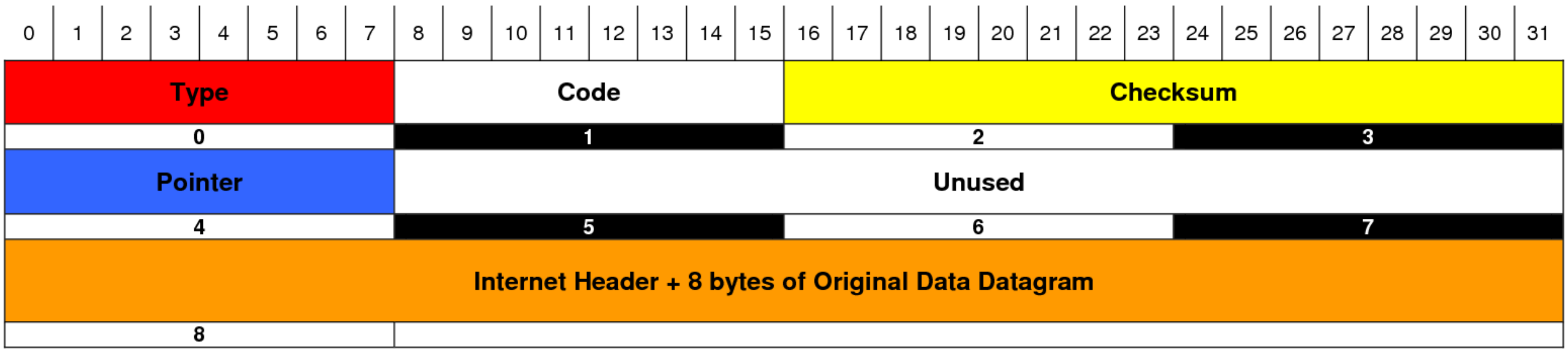
$)

*

Table 9.1 *ICMP messages*

Category	*Type*	*Message*
Error-reporting messages	3	Destination unreachable
	4	Source quench
	11	Time exceeded
	12	Parameter problem
	5	Redirection
Query messages	8 or 0	Echo request or reply
	13 or 14	Timestamp request or reply

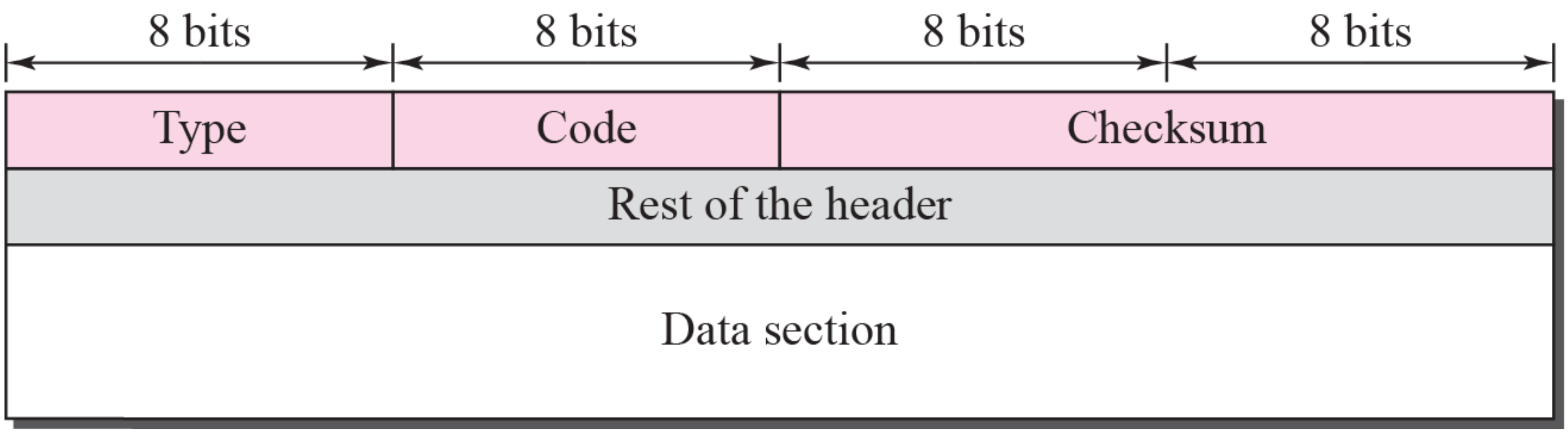
ICMP Parameter Message Format



Type	Code	Meaning
0	0	Echo Reply
3	0	Net Unreachable
	1	Host Unreachable
	2	Protocol Unreachable
	3	Port Unreachable
	4	Frag needed and DF set
	5	Source route failed
	6	Dest network unknown
	7	Dest host unknown
	8	Source host isolated
	9	Network admin prohibited
	10	Host admin prohibited
	11	Network unreachable for TOS
	12	Host unreachable for TOS
	13	Communication admin prohibited
4	0	Source Quench (Slow down/Shut up)

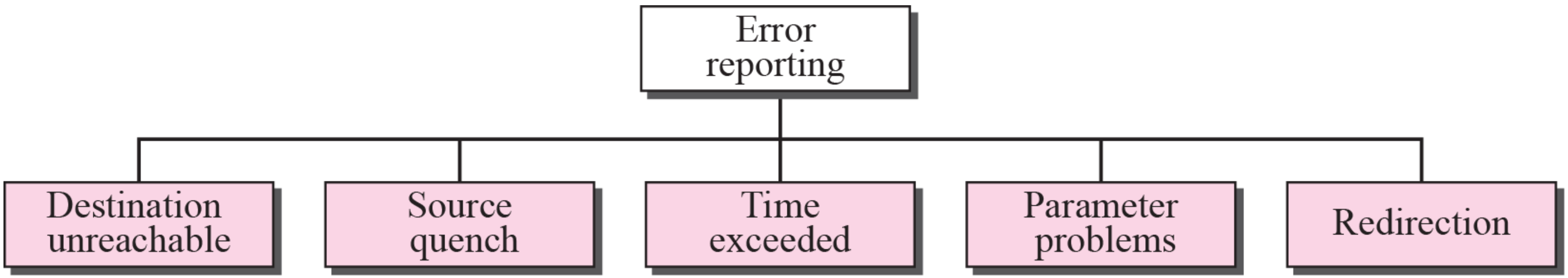
Type	Code	Meaning
5	0	Redirect datagram for the network
	1	Redirect datagram for the host
	2	Redirect datagram for the TOS & Network
	3	Redirect datagram for the TOS & Host
8	0	Echo
9	0	Router advertisement
10	0	Router selection
11	0	Time To Live exceeded in transit
	1	Fragment reassemble time exceeded
12	0	Pointer indicates the error (Parameter Problem)
	1	Missing a required option (Parameter Problem)
	2	Bad length (Parameter Problem)
13	0	Time Stamp
14	0	Time Stamp Reply
15	0	Information Request
16	0	Informaiton Reply
17	0	Address Mask Request
18	0	Address Mask Reply
30	0	Traceroute (Tracert)

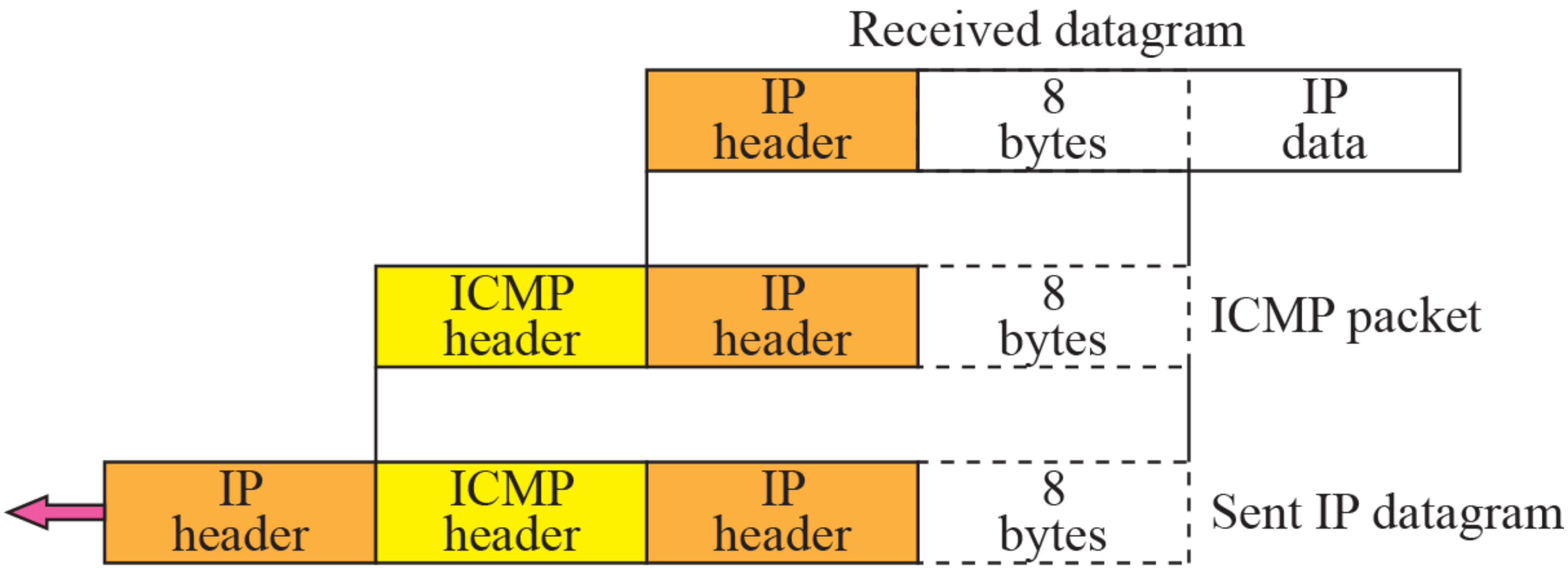
% & +


8 bits
8 bits
8 bits
8 bits
Type
Code
Checksum
Rest of the header
Data section

% & ,

Five types of error




Received datagram
IP header
8 bytes
IP data
ICMP header
IP header
8 bytes
ICMP packet
IP header
ICMP header
IP header
8 bytes
Sent IP datagram

% &.

Type: 3	Code: 0 to 15	Checksum
Unused (All 0s)		
Part of the received IP datagram including IP header plus the first 8 bytes of datagram data		

! "

Source Quench

!

% & / !

Type: 4	Code: 0	Checksum
Unused (All 0s)		
Part of the received IP datagram including IP header plus the first 8 bytes of datagram data		

$!

#

!

$!

Time Exceeded

%

& '

(

%   !

!   !

'

!

(

% & 0 "

Type: 11	Code: 0 or 1	Checksum
Unused (All 0s)		
Part of the received IP datagram including IP header plus the first 8 bytes of datagram data		

(')

! ! &

*

!

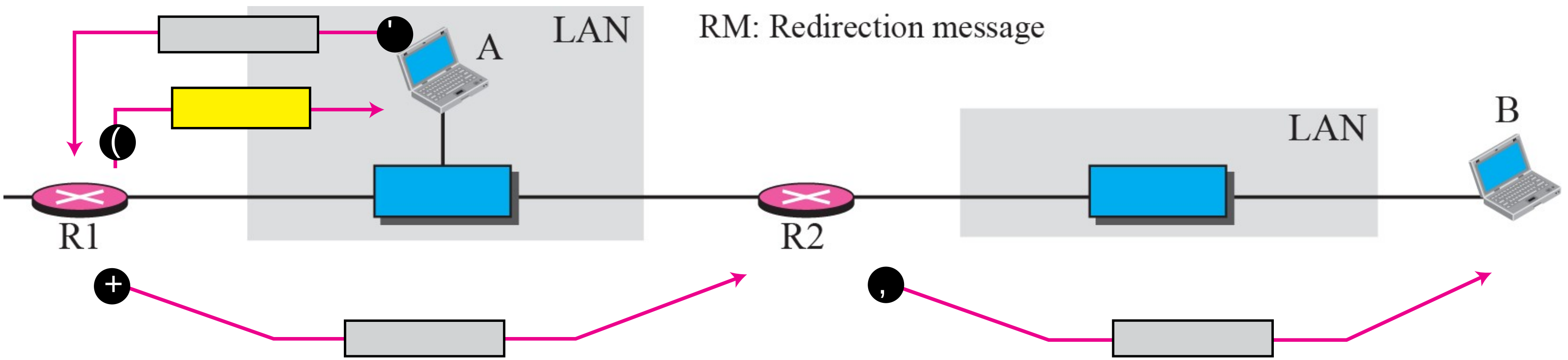
!

Parameter Problem

%   & &

Type: 12	Code: 0 or 1	Checksum
Pointer	Unused (All 0s)	
Part of the received IP datagram including IP header plus the first 8 bytes of datagram data		

% &'1 $



Redirection Message

!

Type: 5	Code: 0 to 3	Checksum
IP address of the target router		
Part of the received IP datagram including IP header plus the first 8 bytes of datagram data		

!

"

Query Messages

Designed for diagnostic Response

Echo Request and Reply

$

$

+ $

"

" !

\+ $

!

\#

"

% & '(!

Type 8: Echo request
Type 0: Echo reply

Type: 8 or 0	Code: 0	Checksum	
Identifier		Sequence number	
Optional data Sent by the request message; repeated by the reply message			

% & '+ " !

Type 13: request
Type 14: reply

Type: 13 or 14	Code: 0	Checksum	
Identifier		Sequence number	
Original timestamp			
Receive timestamp			
Transmit timestamp			

$

!

" &

The timestamp-request and timestamp-reply messages can be used to compute the one-way or round-trip time required for a datagram to go from a source to a destination and then back again. The formulas are

sending time = receive timestamp – original timestamp
receiving time = returned time – transmit timestamp
round-trip time = sending time + receiving time

Note that the sending and receiving time calculations are accurate only if the two clocks in the source and destination machines are synchronized. However, the round-trip calculation is correct even if the two clocks are not synchronized because each clock contributes twice to the round-trip calculation, thus canceling any difference in synchronization.

$

& "

! (

"

$ "" "   %

There are several tools that can be used in the Internet for debugging. We can find if a host or router is alive and running. We can trace the route of a packet. We introduce two tools that use ICMP for debugging: ping and traceroute. We will introduce more tools in future chapters after we have discussed the corresponding protocols.

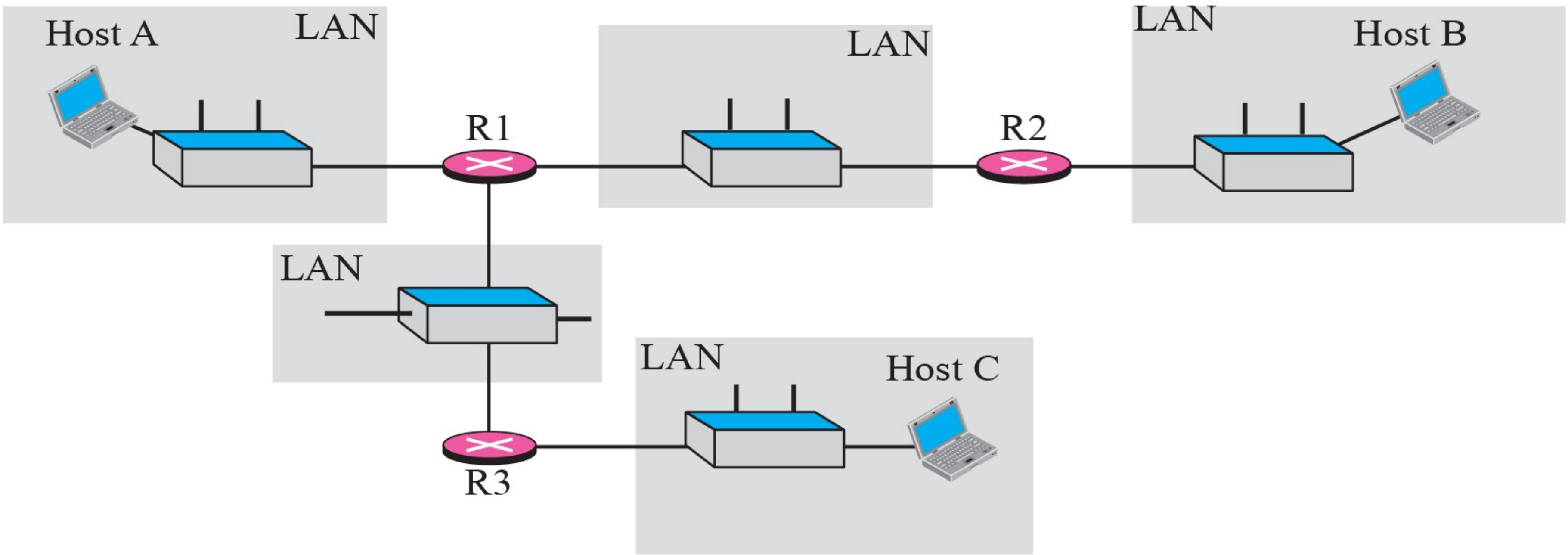
We use the ping program to test the server fhda.edu. The result is shown below:

```
$ ping fhda.edu
PING fhda.edu (153.18.8.1)   56 (84)  bytes of data.
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=0    ttl=62    time=1.91 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=1    ttl=62    time=2.04 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=2    ttl=62    time=1.90 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=3    ttl=62    time=1.97 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=4    ttl=62    time=1.93 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=5    ttl=62    time=2.00 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=6    ttl=62    time=1.94 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=7    ttl=62    time=1.94 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=8    ttl=62    time=1.97 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=9    ttl=62    time=1.89 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=10   ttl=62    time=1.98 ms

--- fhda.edu ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10103 ms
rtt min/avg/max = 1.899/1.955/2.041 ms
```

$ & +

For the second example, we want to know if the adelphia.net mail server is alive and running. The result is shown below: Note that in this case, we sent 14 packets, but only 13 have been returned. We may have interrupted the program before the last packet, with sequence number 13, was returned.

```
$ ping mail.adelphia.net
PING mail.adelphia.net (68.168.78.100) 56(84) bytes of data.
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=0   ttl=48   time=85.4 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=1   ttl=48   time=84.6 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=2   ttl=48   time=84.9 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=3   ttl=48   time=84.3 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=4   ttl=48   time=84.5 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=5   ttl=48   time=84.7 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=6   ttl=48   time=84.6 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=7   ttl=48   time=84.7 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=8   ttl=48   time=84.4 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=9   ttl=48   time=84.2 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=10  ttl=48   time=84.9 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=11  ttl=48   time=84.6 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=12  ttl=48   time=84.5 ms

--- mail.adelphia.net ping statistics ---
14 packets transmitted, 13 received, 7% packet loss, time 13129 ms
rtt min/avg/max/mdev = 84.207/84.694/85.469
```


Host A
LAN
R1
LAN
R2
LAN
Host B
LAN
LAN
Host C
R3

$ & ,

We use the traceroute program to find the route from the computer voyager.deanza.edu to the server fhda.edu. The following shows the result.

```
$ traceroute fhda.edu
traceroute to fhda.edu       (153.18.8.1), 30 hops max, 38 byte packets
1 Dcore.fhda.edu             (153.18.31.25)     0.995 ms     0.899 ms     0.878 ms
2 Dbackup.fhda.edu           (153.18.251.4)     1.039 ms     1.064 ms     1.083 ms
3  tiptoe.fhda.edu           (153.18.8.1)       1.797 ms     1.642 ms     1.757 ms
```

In this example, we trace a longer route, the route to xerox.com. The following is a partial listing.

```
$ traceroute xerox.com
```

traceroute to xerox.com (13.1.64.93), 30 hops max, 38 byte packets

1	Dcore.fhda.edu	(153.18.31.254)	0.622 ms	0.891 ms	0.875 ms
2	Ddmz.fhda.edu	(153.18.251.40)	2.132 ms	2.266 ms	2.094 ms
3	Cinic.fhda.edu	(153.18.253.126)	2.110 ms	2.145 ms	1.763 ms
4	cenic.net	(137.164.32.140)	3.069 ms	2.875 ms	2.930 ms
5	cenic.net	(137.164.22.31)	4.205 ms	4.870 ms	4.197 ms
6	cenic.net	(137.164.22.167)	4.250 ms	4.159 ms	4.078 ms
7	cogentco.com	(38.112.6.225)	5.062 ms	4.825 ms	5.020 ms
8	cogentco.com	(66.28.4.69)	6.070 ms	6.207 ms	5.653 ms
9	cogentco.com	(66.28.4.94)	6.070 ms	5.928 ms	5.499 ms

An interesting point is that a host can send a traceroute packet to itself. This can be done by specifying the host as the destination. The packet goes to the loopback address as we expect.

```
$ traceroute voyager.deanza.edu
traceroute to voyager.deanza.edu   (127.0.0.1), 30 hops max, 38 byte packets
1  voyager          (127.0.0.1)          0.178 ms          0.086 ms          0.055 ms
```

$ & /

Finally, we use the traceroute program to find the route between fhda.edu and mhhe.com (McGraw-Hill server). We notice that we cannot find the whole route. When traceroute does not receive a response within 5 seconds, it prints an asterisk to signify a problem (not the case in this example), and the

```
$ traceroute mhhe.com
traceroute to mhhe.com (198.45.24.104), 30 hops max, 38 byte packets
1   Dcore.fhda.edu       (153.18.31.254)     1.025 ms    0.892 ms    0.880 ms
2   Ddmz.fhda.edu        (153.18.251.40)     2.141 ms    2.159 ms    2.103 ms
3   Cinic.fhda.edu       (153.18.253.126)    2.159 ms    2.050 ms    1.992 ms
4   cenic.net            (137.164.32.140)    3.220 ms    2.929 ms    2.943 ms
5   cenic.net            (137.164.22.59)     3.217 ms    2.998 ms    2.755 ms
6   SanJose1.net         (209.247.159.109)  10.653 ms   10.639 ms   10.618 ms
7   SanJose2.net         (64.159.2.1)       10.804 ms   10.798 ms   10.634 ms
8   Denver1.Level3.net   (64.159.1.114)     43.404 ms   43.367 ms   43.414 ms
9   Denver2.Level3.net   (4.68.112.162)     43.533 ms   43.290 ms   43.347 ms
10  unknown              (64.156.40.134)    55.509 ms   55.462 ms   55.647 ms
11  mcleodusa1.net       (64.198.100.2)     60.961 ms   55.681 ms   55.461 ms
12  mcleodusa2.net       (64.198.101.202)   55.692 ms   55.617 ms   55.505 ms
13  mcleodusa3.net       (64.198.101.142)   56.059 ms   55.623 ms   56.333 ms
14  mcleodusa4.net       (209.253.101.178) 297.199 ms  192.790 ms  250.594 ms
15  eppg.com             (198.45.24.246)    71.213 ms   70.536 ms   70.663 ms
16  ...                  ...                ...         ...         ...
```